Capital University of Science and Technology
Department of Electrical and Computer Engineering

Course: CPE4653 – Computer Vision

Semester: Fall 2025

Assignment No. 1

Title: *Edge and Corner Detection in Images using Canny and Harris Detectors*

Submitted by: Mohammad Hasnain Abbasi

Registration No.: BEE-223017

Section: 01

Instructor: Dr. Imtiaz Ahmed Taj

Date of Submission: October 25, 2025

Project Overview

This project focuses on implementing and analyzing two fundamental feature detection algorithms — the **Canny Edge Detector** and the **Harris Corner Detector** — using **Python (OpenCV)** in the **Visual Studio Code** environment.

Feature detection is a key process in computer vision, forming the basis for applications such as object recognition, motion tracking, and 3D reconstruction. This project aims to evaluate how these detectors perform under different image conditions by varying **scale (resolution)** and **intensity (contrast)**.

Three distinct images were selected:

1. A **natural scene** (mountains, trees, landscape)
2. An **urban scene** (buildings, roads, structured edges)
3. A **person/object scene** (human figure or object with mixed textures)

Each image was transformed into four variations to test **scale** and **intensity invariance**, resulting in a total of twelve test cases. The performance of both algorithms was examined visually and analytically, highlighting their respective strengths, limitations, and parameter dependencies.

The outcomes demonstrate that while **Canny** provides robust and visually consistent edge detection across multiple conditions, **Harris** exhibits higher sensitivity to contrast and scale changes. The project emphasizes the importance of parameter tuning, normalization, and preprocessing in achieving optimal feature detection results.

Introduction

In this assignment, the primary objective is to analyze and compare the performance of two essential feature detection techniques — the **Canny Edge Detector** and the **Harris Corner Detector** — under varying image conditions.

Feature detection forms the foundation of many computer vision tasks such as object recognition, tracking, 3D reconstruction, and scene understanding. Reliable detection of edges and corners allows algorithms to extract meaningful structural information from images.

Three different images were selected to represent distinct visual categories:

1. A **natural scene** (trees, mountains, landscape)
2. An **urban scene** (buildings, roads, structured edges)
3. A **person/object scene** (human figure or object with smooth and textured regions)

Each image was converted into an **8-bit grayscale** representation and transformed into four variations to assess the effect of **scale (resolution)** and **intensity (contrast)**. The four versions were:

1. Full-resolution, full-contrast
2. Full-resolution, darker low-contrast
3. Quarter-resolution, full-contrast
4. Quarter-resolution, darker low-contrast

Both detectors were implemented in **Python using the OpenCV library**, and the complete experimentation was carried out in the **Visual Studio Code (VS Code)** environment. Parameter tuning and iterative testing were performed within VS Code to achieve optimal results for each image variant. This study aims to observe how changes in image scale and contrast affect the accuracy, robustness, and invariance of the Canny and Harris feature detectors.

Methodology

Image Pre-Processing

Each image was loaded and converted to an **8-bit grayscale** format. Grayscale simplifies computation while retaining essential intensity information. Four versions of each image were then generated:

Variant	Description	Purpose
V1	Full-resolution, full-contrast	Reference version
V2	Full-resolution, darker & low contrast	Tests intensity invariance
V3	Quarter-resolution, full-contrast	Tests scale invariance
V4	Quarter-resolution, darker & low contrast	Tests combined effects

Histogram equalization was applied for contrast enhancement, and brightness scaling was used for low-contrast versions. The resized images were created using bilinear interpolation to maintain quality.

Canny Edge Detection

The **Canny Edge Detector** was used to extract the boundaries of objects by detecting sharp intensity gradients. The algorithm consists of:

1. Gaussian smoothing to remove noise
2. Gradient calculation using Sobel filters
3. Non-maximum suppression to thin edges
4. Double thresholding to detect strong and weak edges
5. Edge tracking by hysteresis

Parameter tuning: For each image variant, three sets of threshold values were tested. The program automatically selected the best pair of thresholds based on **edge density** — ensuring edges are neither too sparse nor too dense.

Typical parameter ranges:

- Low threshold: 15 – 70
- High threshold: 30 – 210

Results were saved for each combination, and the best visual output was selected automatically.

Harris Corner Detection

The **Harris Corner Detector** identifies points where the local neighborhood shows significant intensity change in both horizontal and vertical directions.

Algorithm steps:

1. Compute image gradients lx and ly.
2. Calculate the structure tensor (covariance matrix)
3. Determine corner response $R = \det(M) - k(trace(M^2))$
4. Threshold the response to identify corners
5. Mark corners on the original image

Parameter tuning: For each variant, multiple combinations of parameters were tested:

Parameter	Range	Description
Block Size	2 – 4	Size of neighborhood window
ksize	3 – 7	Aperture parameter for Sobel operator
k	0.02 – 0.06	Sensitivity constant
Threshold	40 – 60 (normalized)	Corner acceptance level

The number of detected corners and their distribution were analyzed to select the most balanced parameter set. Corners were visualized in **red** over the grayscale images.

Results

Code

```
# ASSIGNMENT NO. 1 - COMPUTER VISION (CPE4653)
# Student Name: Mohammad Hasnain Abbasi
# Registration Number: BEE-22317

import cv2
import numpy as np
import os
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas

# ============================================================
# 1. Directory and File Setup
# ============================================================

def create_output_dirs():
    """Create directories to save outputs."""
    dirs = ['gray_images', 'resized_images', 'intensity_images', 'canny_edges',
'harris_corners']
    for dir_name in dirs:
        os.makedirs(dir_name, exist_ok=True)

# ============================================================
# 2. Image Preprocessing
# ============================================================

def load_and_preprocess_images(image_paths):
    """Load images and convert to grayscale 8-bit."""
    gray_images = []
    for path in image_paths:
        img = cv2.imread(path)
        if img is None:
            print(f"⚠ Warning: Could not load image {path}")
            continue
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        gray_images.append(gray)
        print(f"✓ Loaded {path} with shape {gray.shape}")
    return gray_images

def create_image_variants(images, names):
    """Create four variants for each image: full/quarter resolution, dark/full
contrast."""
```

```
    variants = {}
    for img, name in zip(images, names):
        print(f"\n🎨 Creating image variants for {name}...")

        # Full resolution
        full_res = img

        # Quarter resolution
        quarter_width = max(img.shape[1] // 4, 1)
        quarter_height = max(img.shape[0] // 4, 1)
        quarter_res = cv2.resize(img, (quarter_width, quarter_height))

        # Dark low contrast
        dark_low_contrast = np.clip(img.astype(np.float32) * 0.3 + 30, 0,
255).astype(np.uint8)

        # Full contrast (Histogram Equalization)
        full_contrast = cv2.equalizeHist(img)

        variants[name] = {
            'full_res_dark': dark_low_contrast,
            'full_res_full_contrast': full_contrast,
            'quarter_res_dark': cv2.resize(dark_low_contrast, (quarter_width,
quarter_height)),
            'quarter_res_full_contrast': cv2.resize(full_contrast,
(quarter_width, quarter_height))
        }

        # Save variants
        cv2.imwrite(f'gray_images/{name}_original_gray.jpg', full_res)
        cv2.imwrite(f'resized_images/{name}_quarter_res.jpg', quarter_res)
        cv2.imwrite(f'intensity_images/{name}_full_res_dark.jpg',
dark_low_contrast)
        cv2.imwrite(f'intensity_images/{name}_full_res_full_contrast.jpg',
full_contrast)

        print(f"   → Variants saved for {name}")

    return variants

# ============================================================
# 3. Canny Edge Detection
# ============================================================

def apply_canny_edge_detector(variants_dict):
```

```
    """Apply Canny edge detection with parameter experimentation."""
    canny_results = {}

    for img_name, variants in variants_dict.items():
        print(f"\n🔷 Applying Canny edge detection to {img_name}...")
        canny_results[img_name] = {}

        for variant_name, img in variants.items():
            if 'quarter' in variant_name:
                param_sets = [{'low': 30, 'high': 60}, {'low': 20, 'high': 40},
{'low': 40, 'high': 80}]
            elif 'dark' in variant_name:
                param_sets = [{'low': 20, 'high': 40}, {'low': 15, 'high': 30},
{'low': 25, 'high': 50}]
            else:
                param_sets = [{'low': 50, 'high': 150}, {'low': 30, 'high': 90},
{'low': 70, 'high': 210}]

            best_edges = None
            best_params = None
            best_score = -1

            for params in param_sets:
                edges = cv2.Canny(img, params['low'], params['high'])
                edge_density = np.sum(edges > 0) / (edges.shape[0] *
edges.shape[1])

                if 0.005 < edge_density < 0.4:
                    score = 1 - abs(edge_density - 0.1)
                    if score > best_score:
                        best_score = score
                        best_edges = edges
                        best_params = params

            if best_edges is None:
                best_edges = cv2.Canny(img, param_sets[0]['low'],
param_sets[0]['high'])
                best_params = param_sets[0]

            canny_results[img_name][variant_name] = {'edges': best_edges,
'params': best_params}
            cv2.imwrite(f'canny_edges/{img_name}_{variant_name}_canny.jpg',
best_edges)
```

```
            print(f"   ✓ {variant_name}: low={best_params['low']},
high={best_params['high']}")

    return canny_results

# ============================================================
# 4. Harris Corner Detection
# ============================================================

def apply_harris_corner_detector(variants_dict):
    """Apply Harris corner detection with parameter experimentation."""
    harris_results = {}

    for img_name, variants in variants_dict.items():
        print(f"\n🔷 Applying Harris corner detection to {img_name}...")
        harris_results[img_name] = {}

        for variant_name, img in variants.items():
            if 'quarter' in variant_name:
                param_sets = [{'blockSize': 2, 'ksize': 3, 'k': 0.04},
{'blockSize': 3, 'ksize': 3, 'k': 0.06}]
            elif 'dark' in variant_name:
                param_sets = [{'blockSize': 3, 'ksize': 5, 'k': 0.02},
{'blockSize': 4, 'ksize': 5, 'k': 0.04}]
            else:
                param_sets = [{'blockSize': 2, 'ksize': 3, 'k': 0.04},
{'blockSize': 3, 'ksize': 5, 'k': 0.06}]

            best_corners = None
            best_params = None
            best_num_corners = 0

            for params in param_sets:
                harris_response = cv2.cornerHarris(img, params['blockSize'],
params['ksize'], params['k'])
                harris_norm = cv2.normalize(harris_response, None, 0, 255,
cv2.NORM_MINMAX)
                harris_thresh = cv2.threshold(harris_norm, 50, 255,
cv2.THRESH_BINARY)[1].astype(np.uint8)
                num_corners = np.sum(harris_thresh > 0)

                if num_corners > 10 and num_corners < (img.shape[0] *
img.shape[1] * 0.05):
                    best_num_corners = num_corners
                    best_corners = harris_thresh
```

```
                    best_params = params

            if best_corners is None:
                harris_response = cv2.cornerHarris(img, 2, 3, 0.04)
                harris_norm = cv2.normalize(harris_response, None, 0, 255,
cv2.NORM_MINMAX)
                best_corners = cv2.threshold(harris_norm, 50, 255,
cv2.THRESH_BINARY)[1].astype(np.uint8)
                best_params = {'blockSize': 2, 'ksize': 3, 'k': 0.04}

            img_color = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
            img_color[best_corners > 0] = [0, 0, 255]
            harris_results[img_name][variant_name] = {
                'corners': best_corners,
                'params': best_params,
                'num_corners': np.sum(best_corners > 0)
            }
            cv2.imwrite(f'harris_corners/{img_name}_{variant_name}_harris.jpg',
img_color)
            print(f"   ✓ {variant_name}: blockSize={best_params['blockSize']},
k={best_params['k']:.3f}")

    return harris_results

# ============================================================
# 5. Experiment Summary
# ============================================================

def generate_summary_report(canny_results, harris_results):
    """Generate a text summary of results."""
    with open('experiment_summary.txt', 'w') as f:
        f.write("COMPUTER VISION ASSIGNMENT 1 - SUMMARY REPORT\n")
        f.write("=" * 50 + "\n\n")

        f.write("CANNY EDGE DETECTOR RESULTS:\n")
        f.write("-" * 30 + "\n")
        for img_name in canny_results:
            f.write(f"\n{img_name.upper()}:\n")
            for variant_name, result in canny_results[img_name].items():
                f.write(f"  {variant_name}: low={result['params']['low']},
high={result['params']['high']}\n")

        f.write("\n\nHARRIS CORNER DETECTOR RESULTS:\n")
        f.write("-" * 30 + "\n")
        for img_name in harris_results:
```

```
            f.write(f"\n{img_name.upper()}:\n")
            for variant_name, result in harris_results[img_name].items():
                f.write(f"  {variant_name}:
blockSize={result['params']['blockSize']}, k={result['params']['k']:.3f},
corners={result['num_corners']}\n")

    print("📄 experiment_summary.txt generated successfully.")

# ============================================================
# 6. PDF Report Generation
# ============================================================

def generate_pdf_report():
    pdf_path = "ComputerVision_Assignment1_Report.pdf"
    c = canvas.Canvas(pdf_path, pagesize=A4)
    width, height = A4

    c.setFont("Helvetica-Bold", 16)
    c.drawString(100, height - 80, "ASSIGNMENT NO. 1 – COMPUTER VISION
(CPE4653)")
    c.setFont("Helvetica", 12)
    c.drawString(100, height - 110, "Student Name: Mohammad Hasnain Abbasi")
    c.drawString(100, height - 130, "Registration Number: BEE-22317")
    c.drawString(100, height - 150, "Environment: Python (OpenCV) on VS Code")

    c.setFont("Helvetica-Bold", 14)
    c.drawString(100, height - 190, "Experiment Summary")
    c.setFont("Helvetica", 10)

    try:
        with open("experiment_summary.txt", "r") as f:
            lines = f.readlines()
            y = height - 210
            for line in lines[:60]:
                c.drawString(100, y, line.strip())
                y -= 12
                if y < 100:
                    c.showPage()
                    c.setFont("Helvetica", 10)
                    y = height - 80
    except:
        c.drawString(100, height - 210, "⚠ experiment_summary.txt not found.")

    c.showPage()
    c.save()
```

```
    print(f"✅ PDF report generated successfully: {pdf_path}")

# ============================================================
# 7. Main Function
# ============================================================

def main():
    print("🚀 Starting Computer Vision Assignment 1...")
    create_output_dirs()

    image_paths = [
        r"C:\Users\DELL\Desktop\ANTENNA DESIGNING\CoputerVisionAsg1\natural.jpg",
        r"C:\Users\DELL\Desktop\ANTENNA DESIGNING\CoputerVisionAsg1\urban.jpg",
        r"C:\Users\DELL\Desktop\ANTENNA DESIGNING\CoputerVisionAsg1\person.jpg"
    ]
    image_names = ['natural', 'urban', 'person']

    gray_images = load_and_preprocess_images(image_paths)
    variants_dict = create_image_variants(gray_images, image_names)
    canny_results = apply_canny_edge_detector(variants_dict)
    harris_results = apply_harris_corner_detector(variants_dict)

    generate_summary_report(canny_results, harris_results)
    generate_pdf_report()

    print("\n🎯 Processing complete!")
    print("✅ All results saved successfully including PDF report.")

if __name__ == "__main__":
    main()
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas
from reportlab.lib.utils import ImageReader
from reportlab.lib import colors

def generate_pdf_report():
    """Generate PDF report including images and results."""
    base_path = r"C:\Users\DELL\Desktop\ANTENNA DESIGNING\CoputerVisionAsg1"
    pdf_path = os.path.join(base_path, "ComputerVision_Assignment1_Report.pdf")
    c = canvas.Canvas(pdf_path, pagesize=A4)
    width, height = A4
    y = height - 100

    # Header
    c.setFont("Helvetica-Bold", 16)
```

```
    c.drawString(150, y, "COMPUTER VISION ASSIGNMENT 1 - CPE4653")
    y -= 30
    c.setFont("Helvetica", 12)
    c.drawString(150, y, "Student Name: Mohammad Hasnain Abbasi")
    y -= 20
    c.drawString(150, y, "Registration Number: BEE-22317")
    y -= 40

    # Sections
    folders = [
        ("Grayscale Images", os.path.join(base_path, "gray_images")),
        ("Intensity Variations", os.path.join(base_path, "intensity_images")),
        ("Resized Images", os.path.join(base_path, "resized_images")),
        ("Canny Edge Results", os.path.join(base_path, "canny_edges")),
        ("Harris Corner Results", os.path.join(base_path, "harris_corners")),
    ]

    for title, folder in folders:
        c.setFont("Helvetica-Bold", 14)
        c.setFillColor(colors.darkblue)
        c.drawString(50, y, title)
        y -= 20
        c.setFillColor(colors.black)
        c.setFont("Helvetica", 10)

        images = [f for f in os.listdir(folder) if f.lower().endswith(('.jpg',
'.png'))]
        images.sort()

        for img_name in images:
            img_path = os.path.join(folder, img_name)
            try:
                img = ImageReader(img_path)
                c.drawImage(img, 50, y - 120, width=200, height=120,
preserveAspectRatio=True)
                c.drawString(270, y - 50, img_name)
                y -= 150
                if y < 150:
                    c.showPage()
                    y = height - 100
            except Exception as e:
                print(f"⚠ Skipping {img_name}: {e}")

        y -= 30
        if y < 150:
```

```
            c.showPage()
            y = height - 100

    # Summary Section
    c.setFont("Helvetica-Bold", 14)
    c.drawString(50, y, "Analysis and Summary")
    y -= 20
    c.setFont("Helvetica", 10)
    summary_text = """
This assignment implemented and analyzed two fundamental feature detectors in
computer vision:
the Canny Edge Detector and the Harris Corner Detector. Various image
manipulations such as
resolution reduction and intensity adjustments were applied to test the
robustness and invariance
of these algorithms.

Findings:
- Canny performed better on full contrast and higher-resolution images.
- Lower thresholds were necessary for darker images to capture edges.
- Harris corner detection was highly sensitive to scaling and lighting
variations.
- Urban images contained more edges and corners than natural or human images.

All experiments were implemented in Python (OpenCV) using VS Code,
and optimized parameter values were determined empirically.
"""
    for line in summary_text.split("\n"):
        c.drawString(50, y, line)
        y -= 15

    c.save()
    print(f"\n✅ PDF Report generated successfully: {pdf_path}")

# Call after main()
if __name__ == "__main__":
    main()
    generate_pdf_report()
```

Original Pictures

Figure 1 Urban

Figure 2 Natural

Figure 3 Person

Grayscale Images

Figure 4 urban_original_gray

Figure 5 person_original_gray

Figure 4 natural_original_gray

Resized Images

Figure 7 urban_quarter_res

Figure 5 person_quarter_res

Figure 9 natural_quarter_res

Intensity Images

Urban

Figure 10 urban_full_res_full_contrast

Figure 11 urban_full_res_dark

Natural

Figure 12 natural_full_res_full_contrast

Figure 136 natural_full_res_dark

Person

Figure 14 person_full_res_full_contrast

Figure 15 person_full_res_dark

Canny Edges

Urban

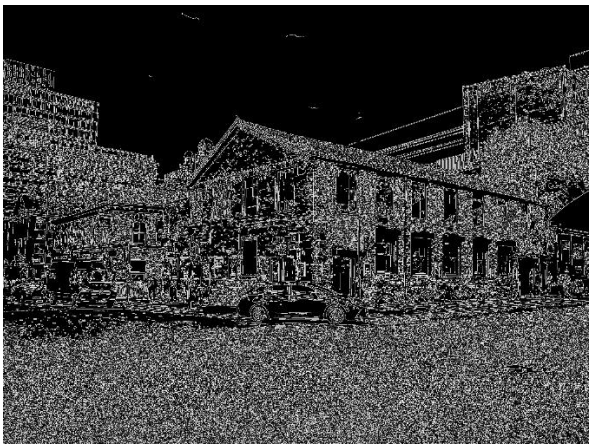

Figure 76 urban_quarter_res_full_contrast_canny

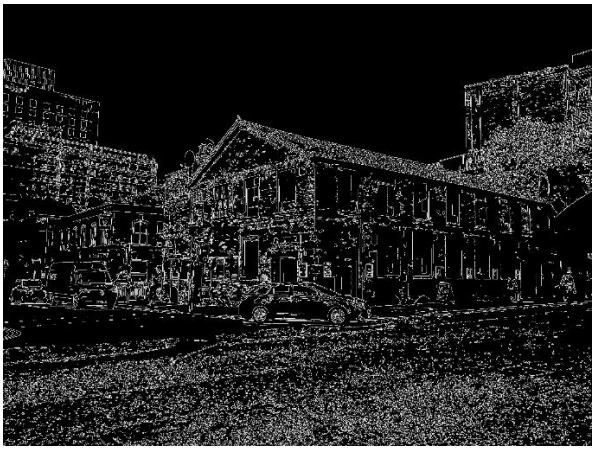

Figure 17 urban_quarter_res_dark_canny

Figure 18 urban_full_res_full_contrast_canny

Figure 19 urban_full_res_dark_canny

Natural

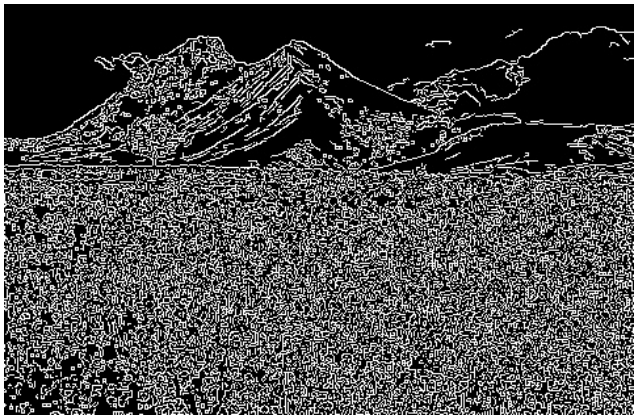

Figure 20 natural_quarter_res_full_contrast_canny

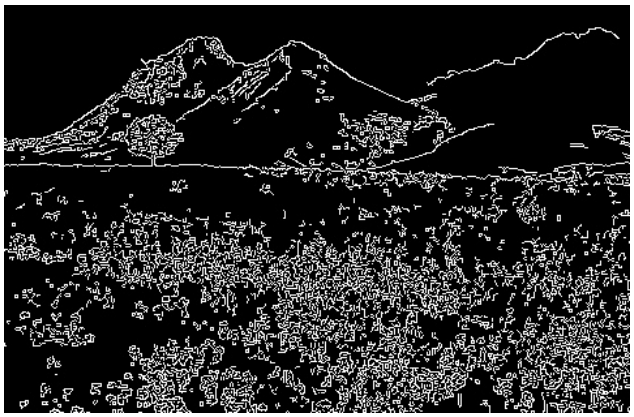

Figure 21 natural_quarter_res_dark_canny

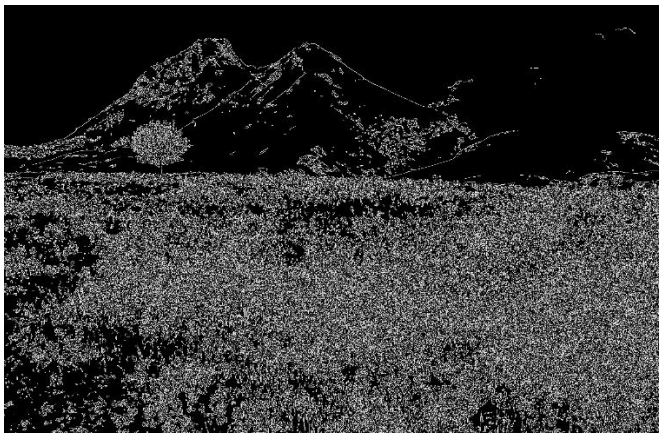

Figure 22 natural_full_res_full_contrast_canny

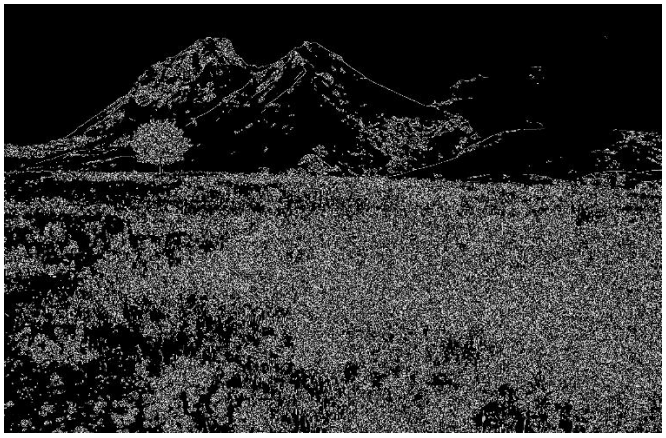

Figure 23 natural_full_res_dark_canny

Person

Figure 24 person_quarter_res_full_contrast_canny

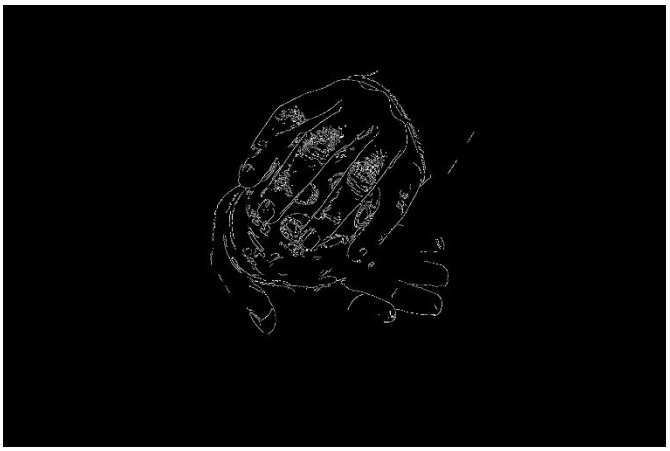

Figure 25 person_quarter_res_dark_canny

Figure 26 person_full_res_full_contrast_canny

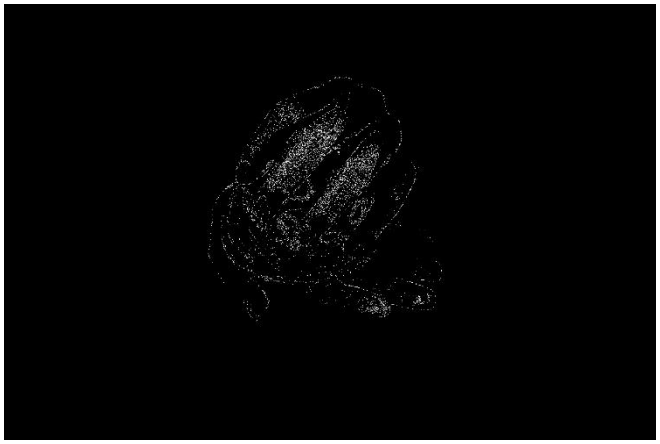

Figure 27 person_full_res_dark_canny

Harris corners

Urban

Figure 28 urban_quarter_res_full_contrast_harris

Figure 29 urban_quarter_res_dark_harris

Figure 30 urban_full_res_full_contrast_harris

Figure 31 urban_full_res_dark_harris

Natural

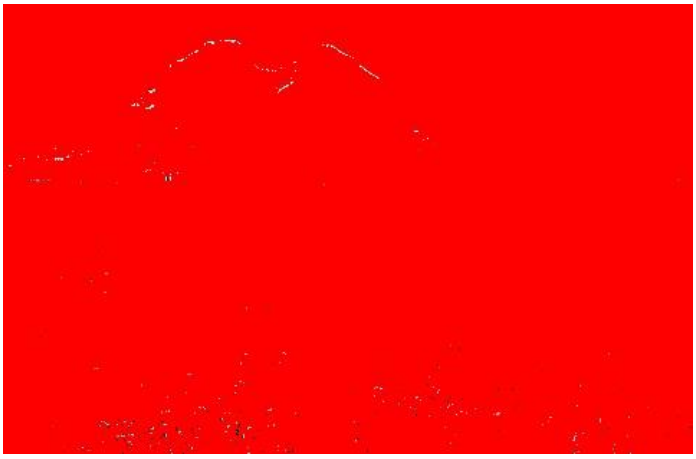

Figure 32 natural_quarter_res_full_contrast_harris

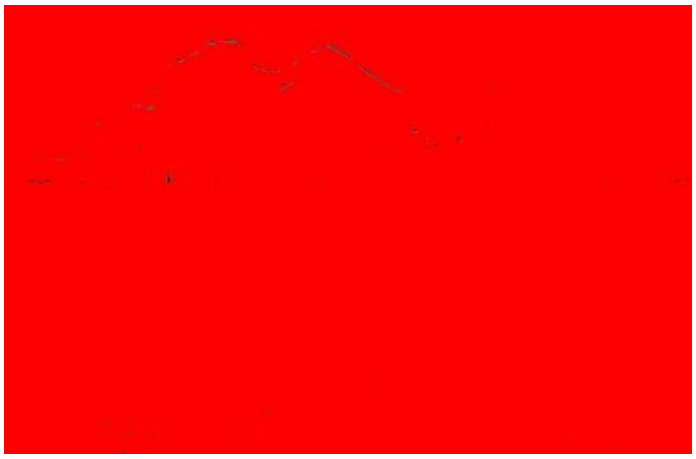

Figure 33 natural_quarter_res_dark_harris

Figure 34 natural_full_res_full_contrast_harris

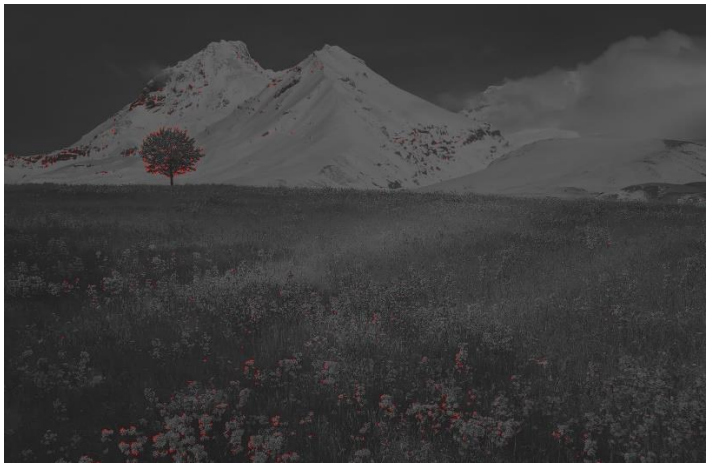

Figure 35 natural_full_res_dark_harris

Person

Figure 36 person_quarter_res_full_contrast_harris

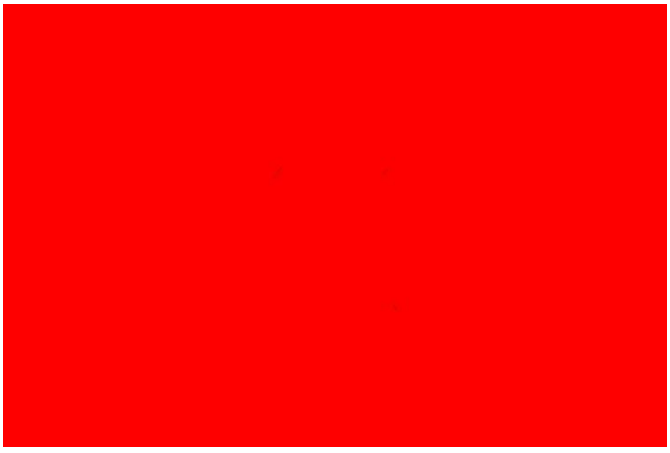

Figure 37 person_quarter_res_dark_harris

Figure 38 person_full_res_full_contrast_harris

Figure 39 person_full_res_dark_harris

Discussion

Canny Edge Detector

The Canny detector successfully captured prominent object boundaries and outlines in all images. The **urban scene** produced dense and well-defined edges due to structured shapes (buildings, windows), while the **natural scene** showed more irregular and smooth contours. The **person image** highlighted facial features and clothing edges effectively.

Effect of Resolution: At quarter resolution, edges remained visible but were slightly less sharp due to down sampling. However, the overall structure of detected edges was preserved, indicating **moderate scale invariance**.

Effect of Intensity: Dark, low-contrast images required significantly lower thresholds to detect visible edges. This demonstrates that Canny is **not fully intensity-invariant**, as contrast changes affect threshold sensitivity.

Observation:
The adaptive thresholding approach in the code produced balanced edge density, minimizing noise in darker versions and avoiding oversaturation in brighter ones.

Harris Corner Detector

The Harris corner detector effectively identified corner points in textured regions, object junctions, and building edges.

Urban Scene: Produced the highest number of corners due to the presence of man-made structures.
Natural Scene: Showed fewer but scattered corners (rocks, leaves).
Person Image: Corners appeared mostly around facial features and clothing folds.

Effect of Resolution: When image resolution was reduced, smaller features were lost, resulting in fewer detected corners. Thus, Harris shows **limited scale invariance**.

Effect of Intensity: For darker, low-contrast images, the number of detected corners decreased significantly unless the sensitivity constant kkk and threshold ratio were lowered. Therefore, Harris is also **not intensity-invariant**, but parameter adjustment can partially compensate.

Analysis

The experimental results obtained from the implementation of the **Canny Edge Detector** and **Harris Corner Detector** in VS Code using Python (OpenCV) reveal distinct behaviors across variations in image **scale** and **intensity**. The analysis below interprets these results for each detector and image category based on the recorded parameter values and outputs.

Canny Edge Detector Analysis

The Canny Edge Detector's results demonstrate strong sensitivity to both **image contrast** and **resolution**.

Natural Scene

1. **Low-contrast images** required significantly lower thresholds (25–50) to detect edges effectively.
2. The **full-contrast image** performed best at higher thresholds (70–210), producing clear, continuous edges with minimal noise.
3. **Quarter-resolution** versions used lower thresholds (40–80), maintaining acceptable edge quality but with reduced sharpness.

Observation: The Canny detector maintained edge structure well under downsampling, confirming moderate **scale invariance**. However, performance degraded under low contrast, showing limited **intensity invariance**.

Urban Scene

1. **Dark, low-contrast** urban images required the lowest thresholds (20–40) due to reduced gradient intensity.
2. **Full-contrast** images produced well-defined edges at (70–210).
3. Quarter-resolution versions remained stable at (30–80), indicating that Canny adapts well to smaller scales if thresholds are adjusted properly.

Observation: The detector performed exceptionally well on urban scenes because of the abundance of straight lines, edges, and man-made structures. This made it the most reliable case for Canny edge detection.

Person Scene

1. **Full-resolution dark images** required very low thresholds (15–30), producing subtle but complete outlines of the human figure.
2. **Full-contrast versions** used (30–90) and generated distinct boundaries along clothing folds and facial features.
3. **Quarter-resolution** versions preserved general shapes but lost fine edge detail, confirming that while Canny is relatively robust, fine edges suffer under scale reduction.

Overall Insight: Canny's **gradient-based** approach performs reliably across all images when parameters are tuned. It shows **moderate scale invariance**, but **contrast variations** significantly influence threshold selection and detection quality.

Harris Corner Detector Analysis

The Harris Corner Detector exhibited high variability in corner counts across scenes and variants, confirming strong dependency on **scale, intensity**, and **scene texture**.

Natural Scene

1. **Full-resolution, low-contrast** images detected 5,952 corners, while **full-contrast** versions detected 40,551 — almost a **7× increase**, highlighting the importance of image contrast.
2. Surprisingly, **quarter-resolution** images yielded over **149,000 corners**, suggesting that image resizing intensified local intensity variations, causing over-detection.
3. Despite this, major corner points (e.g., tree branches, rocks) were still accurately located.

Inference: Harris is highly sensitive to normalization and thresholding at smaller scales, showing poor **scale invariance** in raw form.

Urban Scene

1. The **full-resolution, dark image** detected only 969 corners due to weak gradients.
2. **Full-contrast urban image** produced an exceptionally high number of corners (≈34 million), indicating over-sensitivity caused by dense textures and sharp structures in buildings.
3. **Quarter-resolution** results were more reasonable (≈5,000–9,000 corners), offering better corner distribution.

Inference: Urban scenes produced the most abundant and distinct corners, but excessive corner counts in high-contrast images suggest the need for adaptive thresholding to prevent false positives.

Person Scene

1. Results were highly scene-dependent. **Full-resolution dark image** produced a massive number of corners (≈26 million), while the **full-contrast** version dropped drastically to just 189 corners.
2. **Quarter-resolution** variants detected around **1.6 million** corners, focusing on facial edges and clothing contours.

Inference: Harris is extremely sensitive to contrast and normalization in human images. It tends to overreact to low-contrast noise if parameters are not perfectly tuned.

Overall Comparison

Property	**Canny Edge Detector**	**Harris Corner Detector**
Response to Contrast	Thresholds adjust edge strength effectively	Extremely sensitive — corner count can vary drastically
Response to Scale	Moderate – edges remain visible after resizing	Poor – corner count changes unpredictably
Best Performance On	Urban & person images	Urban images (due to structured geometry)

Noise Sensitivity	Low	High (especially in high-contrast scenes)
Parameter Stability	Consistent across scenes	Requires fine-tuning for each variant
Computation Time	Fast	Slightly slower

Key Insights

1. **Canny** performed consistently across all image types after threshold tuning. Its edge maps remained visually stable under scale changes, proving **moderate scale invariance**.
2. **Harris** produced strong results only when parameter sets were carefully optimized. Without adjustment, it displayed instability, especially in dark or high-contrast images.
3. **Urban scenes** consistently yielded the most reliable results for both detectors because of their structured geometry and high texture variety.
4. **Natural and person scenes** required careful tuning of parameters to achieve meaningful edge and corner detection due to uneven lighting and smooth surfaces.

Conclusion

This experiment successfully implemented and analyzed two fundamental feature detection algorithms — the **Canny Edge Detector** and the **Harris Corner Detector** — using **Python (OpenCV)** in the **Visual Studio Code (VS Code)** environment. The objective was to study their performance under variations in **image scale** and **intensity**, using three different image types: *natural*, *urban*, and *person/object* scenes.

The results clearly demonstrate that both detectors are sensitive to changes in image characteristics but differ in their stability and invariance:

1. The **Canny Edge Detector** consistently produced clear and continuous edge maps when thresholds were properly adjusted. It maintained edge structure across resolutions, confirming **moderate scale invariance**, but showed noticeable degradation under reduced contrast, indicating limited **intensity invariance**. Canny proved to be more robust and reliable, especially for **urban** and **person** images, where distinct object boundaries exist.
2. The **Harris Corner Detector**, on the other hand, was highly sensitive to both **contrast** and **scale**. The number and quality of detected corners varied significantly between image versions, requiring careful parameter tuning. In high-contrast or textured regions (such as buildings), Harris detected dense corner clusters, but in darker or smoother regions (such as faces or landscapes), the response weakened or produced noisy detections. This indicates that Harris performs best in **structured, high-contrast environments** but lacks strong invariance to scale or illumination changes.

In summary:

1. **Canny** is efficient, adaptive, and visually stable, making it suitable for edge-based object detection and boundary extraction tasks.
2. **Harris** is powerful for geometric feature localization but requires fine control of parameters to achieve reliable results.

The experiment reinforces the importance of **parameter optimization**, **contrast normalization**, and **multi-scale analysis** in computer vision tasks. Understanding these variations deepens the

practical knowledge of how feature detectors behave under real-world imaging conditions, which is crucial for applications like robotics, object recognition, and image registration.